Python Programming Practicals

CSC321: Embedded Systems & Internet of Things

Practical Exercises

Instructions: Answer all questions. Write, execute, and test your programs using Python 3. Use appropriate variable names, programming structures, and functions. Handle invalid inputs where required.

1. **Practical 1: Python Fundamentals**
   (a) Using the Python interpreter, execute a statement that displays the following message:

```
Hello World!
```

   (b) Write a Python program that accepts a name from the user and displays a welcome message.
   (c) Consider the following Python statements:

```
x = "45"
y = 10
z = 2.5
```

      i. Determine the data type of each variable.
      ii. Convert `x` to an integer and calculate `x + y`.
      iii. Calculate the value of:

```
y + z * 3
```

      iv. Rewrite the expression using parentheses so that the addition is performed before multiplication.
      v. Swap the values of `y` and `z` using a single Python statement.

   (d) Write a Python program that assigns the values 5 and 3 to variables `a` and `b`, respectively. Display the results of the following operations:
      i. Addition
      ii. Subtraction
      iii. Multiplication
      iv. Division
      v. Integer division
      vi. Modulus
      vii. Exponentiation
   (e) Modify the program to display the results using two decimal places.
   (f) Write a program that accepts the radius of a circle and calculates its area and circumference. Use:

$$A = \pi r^2$$

$$C = 2\pi r$$

   Use the value of π provided by Python's `math` module.

2. **Practical 2: User Input and Error Handling**
   (a) Write a Python program that accepts two floating-point numbers separated by a space. For example:

```
Enter two numbers: 5.5 3.2
```

The program should calculate and display the results of addition, subtraction, multiplication, and division.

(b) Modify the program to handle invalid inputs.
The program must:

i. Accept two numbers from the user.
ii. Detect invalid numerical inputs.
iii. Display an appropriate error message.
iv. Continue requesting input until two valid numbers are entered.
v. Prevent division by zero.

(c) Write a Python program that accepts an integer from the user.
Use exception handling to ensure that the program continues requesting input until a valid integer is entered.

(d) Write a Python function named `read_number()` that:

i. Requests a floating-point number.
ii. Converts the input to a float.
iii. Handles invalid input.
iv. Returns the valid number.

(e) **Arithmetic Expression Parser**
Develop a Python program that accepts arithmetic expressions containing two numbers and one operator.
The supported operators are:

$$+, \quad -, \quad \times, \quad \div$$

Examples include:

```
5.3+4.7
10 - 3
-1--1
.3+.7
```

Your program should identify the operands and operator, perform the calculation, and display the result.
Test the parser using the following inputs:

Input	Expected Result
`5.3+4.7`	Valid
`5.3++4.7`	Invalid
`5..3 + 4.7`	Invalid
`.3+.7`	Valid
`5.3+4b7`	Invalid
Empty input	Invalid
`5.3+.`	Invalid
`5.3+`	Invalid
`-1-1`	Valid
`-1-1`	Invalid

Test expressions containing multiple spaces.
Hint: You may use Python's `re` module for regular expression matching.

3. **Practical 3: Conditional Statements and Loops**

(a) Write a Python program that assigns the values 5 and 7 to variables `a` and `b`.
Compare the values and display the larger number.
Modify the program to accept two integers from the user and handle invalid inputs.

(b) **Temperature Classification**
A temperature monitoring application receives readings in degrees Celsius.
Write a Python program that classifies the temperature as follows:

Temperature	Status
Below $20°C$	LOW
$20°C$ to $30°C$	NORMAL
Above $30°C$	HIGH

The program should accept a temperature from the user and display the corresponding status.

(c) Write a Python program that prints all even numbers between 1 and 50.

(d) Using a `while` loop, print the numbers from 10 down to 1.

(e) Given the following list:

```
L = [3, 5, 7, 9, 11]
```

Write a program that calculates the sum of the elements without using `sum()`.

(f) Write a Python program that accepts a positive integer n and calculates:

$$S = 1 + 2 + 3 + \cdots + n$$

Use a loop rather than a mathematical formula.

(g) Write a program that continuously accepts temperature readings until the user enters the value `-1`. The program should display:

i. Number of valid readings.
ii. Average temperature.
iii. Highest temperature.
iv. Lowest temperature.

The value `-1` must not be included in the calculations.
Handle the case where no valid reading has been entered.

4. **Practical 4: Functions and Data Structures**

(a) Write a Python function named `square(x)` that accepts a number and returns its square.

(b) Write a function named `calculate_area()` that accepts the length and width of a rectangle and returns its area.

(c) Write a function that accepts a list of numbers and returns:

i. The largest number.
ii. The smallest number.
iii. The average value.

Do not use the built-in functions `max()`, `min()`, or `sum()`.

(d) Create a Python dictionary representing a student with the following information:

i. Name
ii. Age
iii. Programme

Add a new key named `GPA` with the value `3.5`.
Display the complete dictionary.

(e) Create a list containing five student dictionaries.
Each dictionary should contain:

i. Student name
ii. Registration number
iii. Examination score

Write a program that displays the names and scores of all students.

(f) Write a function named `classify_temperature(temp)` that accepts a temperature value and returns `LOW`, `NORMAL`, or `HIGH` using the ranges defined in Practical 3.

(g) Create a list containing five temperature readings.
Use a loop to call `classify_temperature()` for each reading and display the results.

(h) Write a function named `count_status(readings)` that accepts a list of temperature readings and returns the number of readings classified as:

i. LOW
ii. NORMAL
iii. HIGH

5. **Practical 5: Mathematical Functions and Number Series**

(a) **Sine Function**

Write a Python program that accepts an angle in degrees and calculates its sine.

Use the `math` module.

Test your program using:

$$0^\circ, \quad 30^\circ, \quad 45^\circ, \quad 60^\circ, \quad 90^\circ$$

Explain why the angle must be converted from degrees to radians.

(b) **Fibonacci Series**

The Fibonacci sequence is defined as:

$$F(0) = 0, \qquad F(1) = 1$$

$$F(n) = F(n-1) + F(n-2), \qquad n \geq 2$$

Write a Python program that accepts a non-negative integer n and displays the Fibonacci numbers from $F(0)$ to $F(n)$.

Test the program using:

$$n = 0,\ 1,\ 2,\ 3,\ 20$$

(c) Modify the Fibonacci program to accept a maximum value and display all Fibonacci numbers smaller than that value.

(d) **Prime Numbers**

Write a Python function that accepts a positive integer n and returns all prime numbers less than or equal to n.

(e) **Factorial**

The factorial of a non-negative integer is defined as:

$$n! = \prod_{k=1}^{n} k$$

where:

$$0! = 1$$

Write a Python function that calculates the factorial of a non-negative integer.

(f) **Geometric Series**

The sum of the first n terms of a geometric series is:

$$S_n = \sum_{k=0}^{n-1} ar^k$$

Write a function that accepts a, r, and n and calculates S_n.

(g) **Harmonic Series**

The nth harmonic number is:

$$H_n = \sum_{k=1}^{n} \frac{1}{k}$$

Write a Python function that calculates the harmonic number H_n.

(h) **Series Comparison**

Write a Python program that calculates and displays the first ten terms of:

i. Fibonacci sequence.
ii. Geometric sequence with $a = 1$ and $r = 2$.
iii. Harmonic sequence H_n.

Compare the growth of the three sequences.

6. **Practical 6: Classes, Objects, and Python Modules**

Create a Python class named `MathSeries` that provides methods for calculating mathematical number series.

(a) Implement the following methods:

i. `fibonacci(n)`
Returns Fibonacci numbers from $F(0)$ to $F(n)$.
ii. `fibonacci_max(max_value)`
Returns Fibonacci numbers smaller than `max_value`.
iii. `primes(max_value)`
Returns all prime numbers less than or equal to `max_value`.
iv. `factorial(n)`
Returns the factorial of n.
v. `geometric(a, r, n)`
Returns the sum of the first n terms of a geometric series.
vi. `harmonic(n)`
Returns the nth harmonic number.

(b) Save the class in a Python module named:

```
mathSeries.py
```

(c) Create another Python file named:

```
main.py
```

Import the module and create an instance of the `MathSeries` class.
For example:

```
import mathSeries

series = mathSeries.MathSeries()
```

Call each method and display the results.

(d) Modify the `MathSeries` class to handle invalid parameters.
For example:

i. Negative factorial arguments.
ii. Negative Fibonacci indices.
iii. Invalid numbers of terms.

Use appropriate Python exceptions.

(e) **Plotting Mathematical Series**

Using Python's Matplotlib library, write a program that plots:

i. Fibonacci numbers.
ii. Harmonic numbers.
iii. Geometric sequence.

Each plot should have:

i. A title.
ii. Labels for both axes.
iii. Appropriate data points.

7. **Practical 7: Object-Oriented Programming**

Scenario:

You are developing a Python application that simulates devices in a smart home.

Each device has a unique identifier, a name, and an operational status.

Some devices provide simulated sensor readings.

(a) **Base Device Class**

Create a class named `Device` with the following attributes:

i. `device_id`
ii. `name`
iii. `status`

The status must be either `"online"` or `"offline"`.

Implement the following methods:

i. `toggle()`
Changes the device status between online and offline.
ii. `info()`
Returns the device information.

(b) **Temperature Sensor**

Create a subclass named `TemperatureSensor` that inherits from `Device`.

Add an attribute named `temperature`.

Implement a method named `read()` that returns the current temperature.

(c) **Motion Sensor**

Create another subclass named `MotionSensor`.

Add an attribute named `motion_detected`.

This attribute must contain either `True` or `False`.

Implement a method named `read()` that returns the current motion detection status.

(d) Create instances of both sensor classes.

Test the following operations:

i. Display device information.
ii. Read sensor values.
iii. Change device status.
iv. Display the updated status.

(e) **Simulated Sensor Readings**

Modify the sensor classes to generate simulated readings using Python's `random` module.

The temperature sensor should generate values between $15°C$ and $40°C$.

The motion sensor should randomly return `True` or `False`.

(f) Write a program that creates three temperature sensors and two motion sensors.

Generate and display readings from all five sensors.